Könyvtári kölcsönzési szolgáltatás

Öt adatbázis, egy döntés – 3–4 perces magyarázat (felolvasható szöveg)

Adatbázis	Mit tárol	Miért itt?
PostgreSQL	tagok, készlet, kölcsönzések, döntések	tranzakció, kényszer, sorzár
MongoDB	katalógusrekord (szabálytalan mezők)	rugalmas, ellenőrzött dokumentum
Redis	döntések gyorsítótára (TTL 5 perc)	gyors, magától lejár
Neo4j	szerző, tárgykör, ki mit kölcsönzött	kapcsolatok bejárása
pgvector + Qdrant	leírások vektorai (420 db)	hasonlóság-keresés, összehasonlítás

1. Mi ez az alkalmazás? (0:00 – 0:30)

Egy könyvtári kölcsönzési szolgáltatást készítettem. Ha egy tag kölcsönözni szeretne egy könyvet, a program három válasz közül dönt: JÓVÁHAGYVA, VÁRÓLISTÁRA kerül, ha nincs szabad példány, vagy ELUTASÍTVA, ha valamelyik szabály sérül, például a tag fel van függesztve, tartozik, már három könyve van nála, vagy túl fiatal a könyvhöz. Ha a válasz nem igen, a program alternatív könyveket ajánl. A feladat az volt, hogy mind az öt adatbázistípust használjam, és mindegyik azt tárolja, amihez a legjobban illik.

2. Az öt adatbázis szerepe (0:30 – 1:40)

A PostgreSQL a tagokat, a készletet, a kölcsönzéseket és a döntéseket tárolja, mert itt kell a tranzakció és a kényszer. Ha ketten egyszerre kérik az utolsó példányt, a sorzár miatt az egyik megkapja, a másik várólistára kerül. A MongoDB a könyvek katalógusrekordját tartalmazza, mert a valódi rekordok szabálytalanok: a regényeknél a szereplők, az ismeretterjesztő könyveknél a helyszínek kötelezők. A séma-ellenőrzés a hiányos beszúrást hibával visszautasítja. A Redis gyorsítótár: a döntést öt percig tárolja. Ha felfüggesztünk egy tagot, de a kulcsot nem töröljük, a cache még mindig igent mond, ezért a módosítással egy lépésben törlöm a kulcsot. A Neo4j a kapcsolatokat tárolja: szerző, tárgykörök, és hogy ki mit kölcsönzött még. A pgvector és a Qdrant a könyvleírások beágyazását tárolja, a hasonló könyvek kereséséhez. Szándékosan kétszer építettem meg, hogy összehasonlíthassam őket. A PostgreSQL és a MongoDB az igazság forrása, a többi csak származtatott másolat.

3. Az adatok (1:40 – 2:05)

A 420 könyv valódi: az Open Library nyílt katalógusából töltöttem le, címmel, szerzővel, leírással és ISBN-nel. A tagok, a kölcsönzések, a példányszámok és a korhatárok szintetikusak, mert a valódi kölcsönzési adatok személyes adatok, és nem nyilvánosak. Az életkort a teljes születési dátumból számolom: aki holnap lesz tizennyolc, az ma még tizenhét. A 19 tesztesetet a kód előtt írtam meg, és mind sikeres.

4. Vektorkeresés: pgvector és Qdrant (2:05 – 3:00)

Mindkét rendszerbe ugyanazok a normalizált, 384 dimenziós vektorok kerültek, koszinusz-távolsággal, ugyanazzal a húsz lekérdezéssel. A közelítő HNSW-keresés mindkettőnél pontosan megegyezett a pontos kereséssel: az ANN recall@10 értéke 1,0, a két rendszer tízes listája pedig mind a tíz azonosítóban egyezett. Ez 420 vektornál várható, mert az index gyakorlatilag pontos. A címkézett minőség alacsonyabb, recall@5 0,58 és MRR 0,775, mert a műfajcímkék az Open Library közösségi címkéiből jönnek, és zajosak. Az egyetlen SQL-lekérdezés, amely a vektorkeresést a készlettel összekapcsolja, pgvectorban egy utasítás, Qdrantnál két lépés. Az eredmény ugyanaz.

5. A hibaeset (3:00 – 3:30)

A legérdekesebb hiba a szűrt keresés. Ha a keresést egy olyan műfajra szűröm, amely a sorok körülbelül öt százalékát adja, a pgvector alapbeállítással átlagosan csak 1,65 sort ad vissza tíz helyett. Az index csak negyven jelöltet ad, és a szűrés ezután fut. Ha bekapcsolom az iteratív keresést, mind a tíz sor megjön. A Qdrant payload-indexszel alapból jól működik.

6. Összefoglalás (3:30 – 3:50)

Egy forrásrekord módosítása után a Redis, a vektor és a gráf addig elavult, amíg frissítést nem futtatunk; ezt a program észleli és javítja. Két saját hibámat is javítottam és dokumentáltam: egy gyorsítótárazott lekérdezési tervet, ami meghamisította a mérést, és egy rossz első adatletöltést. Minden szám a kód kimenetéből származik.